

16-bit RISC Processor

Single-Cycle Datapath Implementation in Verilog HDL

Madhav Purohit

B.Tech, Electronics and Communication Engineering

National Institute of Technology (NIT) Hamirpur

VLSI Domain | Apsis Solutions | 1Stop Training & Placement Program

1. Design

This processor implements a minimal 16-bit RISC instruction set on a single-cycle datapath: every instruction is fetched, decoded, executed, and written back within one clock cycle. The design uses 8 general-purpose registers (R0-R7), a 16-bit ALU, separate instruction and data memories (Harvard-style), and a control unit that decodes a 4-bit opcode into datapath control signals.

1.1 Instruction Set Architecture

Every instruction is 16 bits wide and follows one of three formats, distinguished by the 4-bit opcode:

Format	Bit layout	Used by
R-type	opcode[15:12] rs[11:9] rt[8:6] rd[5:3] unused[2:0]	ADD, SUB, AND, OR
I-type	opcode[15:12] rs[11:9] rd/rt[8:6] imm6[5:0]	ADDI, LOAD, STORE, BEQ
J-type	opcode[15:12] addr[11:0]	JMP, HALT

Note on the I-type field at bits [8:6]: for ADDI/LOAD it is the destination register; for STORE it is the source register whose value is written to memory; for BEQ it is the second register being compared. Because this field sits at the same bit position as the R-type "rt" field, the register file's existing rt read port supplies the correct operand for all four I-type instructions without any extra hardware.

Opcode	Mnemonic	Operation
0000	ADD rd, rs, rt	$rd = rs + rt$
0001	SUB rd, rs, rt	$rd = rs - rt$
0010	AND rd, rs, rt	$rd = rs \& rt$
0011	OR rd, rs, rt	$rd = rs rt$
0100	ADDI rd, rs, imm6	$rd = rs + \text{sign_extend}(\text{imm6})$
0101	LOAD rd, rs, imm6	$rd = \text{DMEM}[rs + \text{sign_extend}(\text{imm6})]$
0110	STORE rd, rs, imm6	$\text{DMEM}[rs + \text{sign_extend}(\text{imm6})] = rd$
0111	BEQ rs, rd, imm6	if $(rs == rd)$ $PC = PC + 1 + \text{sign_extend}(\text{imm6})$
1000	JMP addr12	$PC = \text{addr12}$
1111	HALT	processor stops (PC frozen)

1.2 Datapath Block Diagram

The block diagram below shows how instruction fetch, register access, ALU execution, data memory access, and the next-PC calculation connect together in the single-cycle datapath.

16-bit RISC Processor — Single-Cycle Datapath Block Diagram

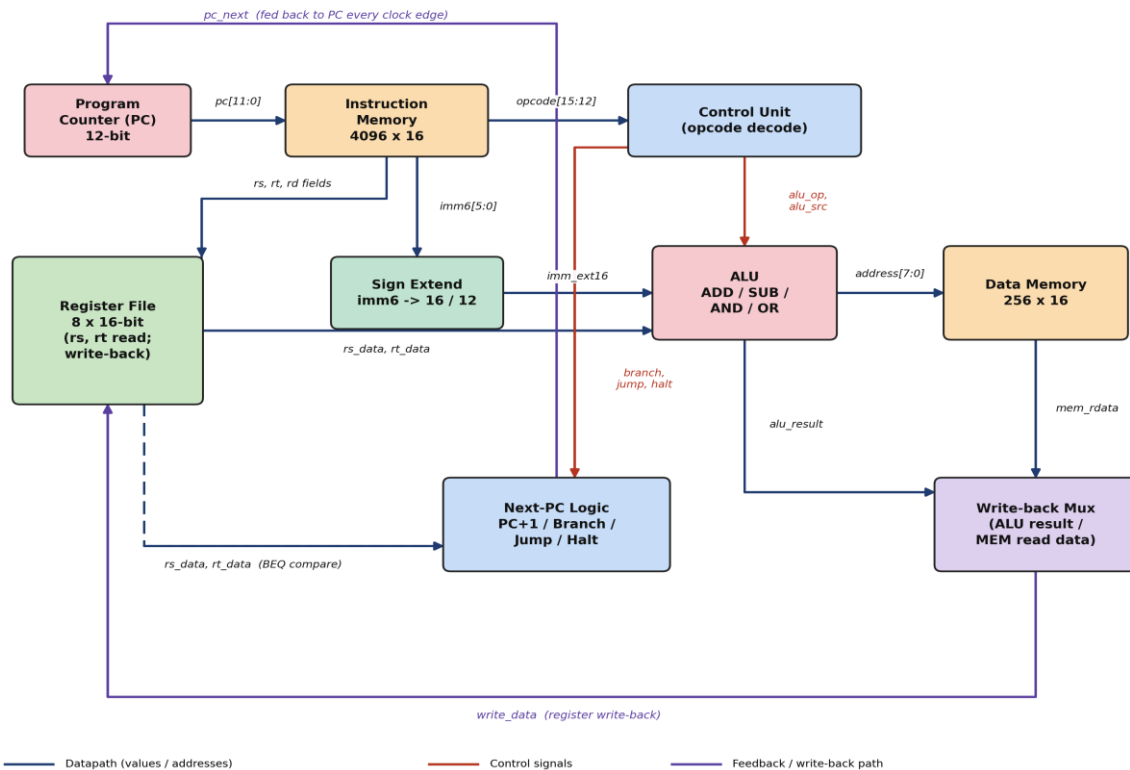


Figure 1: Single-cycle datapath block diagram

1.3 ALU -- RTL Design

```
`timescale 1ns/1ps
module alu (
    input  [15:0] a,
    input  [15:0] b,
    input  [1:0]  alu_op,    // 00=ADD, 01=SUB, 10=AND, 11=OR
    output reg [15:0] result
);
    always @(*) begin
        case (alu_op)
            2'b00: result = a + b;
            2'b01: result = a - b;
            2'b10: result = a & b;
            2'b11: result = a | b;
            default: result = 16'h0000;
        endcase
    end
endmodule
```

1.4 Register File -- RTL Design

```
`timescale 1ns/1ps
module register_file (
    input  clk,
    input  rst_n,
    input  reg_write,
    input  [2:0] rs_addr,
    input  [2:0] rt_addr,
```

```

    input  [2:0] write_addr,
    input  [15:0] write_data,
    output [15:0] rs_data,
    output [15:0] rt_data
);
    reg [15:0] regs [0:7];
    integer i;

    assign rs_data = regs[rs_addr];
    assign rt_data = regs[rt_addr];

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            for (i = 0; i < 8; i = i + 1)
                regs[i] <= 16'h0000;
            end else if (reg_write) begin
                regs[write_addr] <= write_data;
            end
        end
    end
endmodule

```

1.5 Control Unit -- RTL Design

```

`timescale 1ns/1ps
module control_unit (
    input  [3:0] opcode,
    output reg reg_write,
    output reg mem_read,
    output reg mem_write,
    output reg alu_src,      // 0 = second ALU operand is rt, 1 = sign-extended immediate
    output reg branch,
    output reg jump,
    output reg halt,
    output reg mem_to_reg,  // 1 = write-back value comes from data memory (LOAD)
    output reg [1:0] alu_op
);
    // Opcode map
    localparam ADD    = 4'b0000;
    localparam SUB    = 4'b0001;
    localparam AND_OP = 4'b0010;
    localparam OR_OP  = 4'b0011;
    localparam ADDI   = 4'b0100;
    localparam LOAD   = 4'b0101;
    localparam STORE  = 4'b0110;
    localparam BEQ    = 4'b0111;
    localparam JMP    = 4'b1000;
    localparam HALT   = 4'b1111;

    always @(*) begin
        // Safe defaults every cycle (avoids unintended latches)
        reg_write  = 1'b0;
        mem_read   = 1'b0;
        mem_write  = 1'b0;
        alu_src    = 1'b0;
        branch     = 1'b0;
        jump       = 1'b0;
        halt       = 1'b0;
        mem_to_reg = 1'b0;
        alu_op     = 2'b00;

        case (opcode)
            ADD:  begin reg_write = 1'b1; alu_op = 2'b00; end
            SUB:  begin reg_write = 1'b1; alu_op = 2'b01; end
            AND_OP: begin reg_write = 1'b1; alu_op = 2'b10; end
        endcase
    end
endmodule

```

```

        OR_OP: begin reg_write = 1'b1; alu_op = 2'b11; end
        ADDI: begin reg_write = 1'b1; alu_src = 1'b1; alu_op = 2'b00; end
        LOAD: begin reg_write = 1'b1; alu_src = 1'b1; alu_op = 2'b00;
                mem_read = 1'b1; mem_to_reg = 1'b1; end
        STORE: begin alu_src = 1'b1; alu_op = 2'b00; mem_write = 1'b1; end
        BEQ:  begin branch = 1'b1; end
        JMP:  begin jump = 1'b1; end
        HALT: begin halt = 1'b1; end
        default: ; // treated as NOP
    endcase
end
endmodule

```

1.6 Top-Level Datapath -- RTL Design

The top-level module fetches each instruction, decodes its fields, and wires the register file, ALU, and data memory together, computing the next PC value every cycle.

```

`timescale 1ns/1ps
module riscv16_top (
    input clk,
    input rst_n,
    output [11:0] pc_out,    // exposed for observability in simulation
    output halted
);
    // ----- Instruction fetch -----
    reg [11:0] pc;
    wire [11:0] pc_next;
    reg [15:0] imem [0:4095]; // instruction memory, preloaded by the testbench
    wire [15:0] instr = imem[pc];

    // ----- Instruction fields -----
    wire [3:0] opcode    = instr[15:12];
    wire [2:0] rs_addr   = instr[11:9];
    wire [2:0] rt_addr   = instr[8:6]; // R-type "rt", and I-type dest/src/compare reg (same bit
slot)
    wire [2:0] rd_addr_r = instr[5:3]; // R-type destination only
    wire [5:0] imm6      = instr[5:0];
    wire [11:0] jaddr    = instr[11:0];

    wire [15:0] imm_ext16 = {{10{imm6[5]}}, imm6}; // sign-extend to 16 bits (ALU operand)
    wire [11:0] imm_ext12 = {{6{imm6[5]}}, imm6}; // sign-extend to 12 bits (branch offset)

    // ----- Control unit -----
    wire reg_write, mem_read, mem_write, alu_src, branch, jump, halt, mem_to_reg;
    wire [1:0] alu_op;
    control_unit cu (
        .opcode(opcode), .reg_write(reg_write), .mem_read(mem_read),
        .mem_write(mem_write), .alu_src(alu_src), .branch(branch),
        .jump(jump), .halt(halt), .mem_to_reg(mem_to_reg), .alu_op(alu_op)
    );

    // R-type instructions (ADD/SUB/AND/OR) write to instr[5:3];
    // I-type instructions (ADDI/LOAD) write to instr[8:6].
    wire is_r_type = (opcode == 4'b0000) || (opcode == 4'b0001) ||
                    (opcode == 4'b0010) || (opcode == 4'b0011);
    wire [2:0] write_addr = is_r_type ? rd_addr_r : rt_addr;

    // ----- Register file -----
    wire [15:0] rs_data, rt_data;
    wire [15:0] write_back_data;
    register_file rf (
        .clk(clk), .rst_n(rst_n), .reg_write(reg_write),

```

```

        .rs_addr(rs_addr), .rt_addr(rt_addr), .write_addr(write_addr),
        .write_data(write_back_data),
        .rs_data(rs_data), .rt_data(rt_data)
    );

    // ----- ALU -----
    wire [15:0] alu_b = alu_src ? imm_ext16 : rt_data;
    wire [15:0] alu_result;
    alu alu_inst (
        .a(rs_data), .b(alu_b), .alu_op(alu_op), .result(alu_result)
    );

    // ----- Data memory -----
    // 256 x 16-bit, addressed by the lower 8 bits of the computed address
    reg [15:0] dmem [0:255];
    wire [7:0] dmem_addr = alu_result[7:0];
    wire [15:0] dmem_rdata = dmem[dmem_addr];

    // STORE writes the register in the "rt" bit-slot (instr[8:6]) to memory
    always @(posedge clk) begin
        if (mem_write)
            dmem[dmem_addr] <= rt_data;
    end

    assign write_back_data = mem_to_reg ? dmem_rdata : alu_result;

    // ----- Branch / jump / halt resolution -----
    wire branch_taken = branch && (rs_data == rt_data);
    wire [11:0] pc_plus1 = pc + 12'd1;
    wire [11:0] branch_target = pc_plus1 + imm_ext12;

    assign pc_next = halt
        ? pc :
        jump
        ? jaddr :
        branch_taken ? branch_target :
        pc_plus1;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) pc <= 12'd0;
        else pc <= pc_next;
    end

    assign pc_out = pc;
    assign halted = halt;
endmodule

```

2. Testbench

2.1 Test Program

The testbench preloads instruction memory with a short program that exercises every instruction in the ISA at least once: two ADDI loads set up initial values, ADD/SUB/AND/OR combine them, STORE writes a result to data memory, LOAD reads it back, BEQ takes a branch to skip two dead-code instructions, JMP skips a further dead-code instruction, and HALT stops the processor.

Addr	Instruction	Effect
0	ADDI R1, R0, 5	R1 = 5
1	ADDI R2, R0, 3	R2 = 3

Addr	Instruction	Effect
2	ADD R3, R1, R2	R3 = 8
3	SUB R4, R1, R2	R4 = 2
4	AND R5, R1, R2	R5 = 1
5	OR R6, R1, R2	R6 = 7
6	STORE [R0+10], R3	DMEM[10] = 8
7	LOAD R7, R0, 10	R7 = 8
8	BEQ R3, R7, 2	R3==R7 -> PC = 11 (skips addr 9-10)
9-10	ADDI R1, ... (dead code)	must NOT execute
11	JMP 13	PC = 13 (skips addr 12)
12	ADDI R1, ... (dead code)	must NOT execute
13	HALT	processor stops

This program was first traced by hand against the ISA table above to compute the expected final register and memory values independently of the RTL; those values are what the testbench's self-checks below verify against.

2.2 Testbench Code

```
`timescale 1ns/1ps
module riscp16_tb;
    reg clk;
    reg rst_n;
    wire [11:0] pc_out;
    wire        halted;

    integer errors;
    integer i;

    riscp16_top dut (
        .clk(clk), .rst_n(rst_n),
        .pc_out(pc_out), .halted(halted)
    );

    initial clk = 0;
    always #5 clk = ~clk;

    task check16; input [15:0] actual; input [15:0] expected; input [8*32:1] msg; begin
        if (actual !== expected) begin
            $display("*** CHECK FAILED: %s (expected %0d, got %0d) ***", msg, expected, actual);
            errors = errors + 1;
        end else begin
            $display(" CHECK PASSED: %s = %0d", msg, actual);
        end
    end endtask

    initial begin
        errors = 0;

        // ---- Preload instruction memory with the test program ----
```

```

// addr0: ADDI R1, R0, 5
dut.imem[0] = 16'h4045;
// addr1: ADDI R2, R0, 3
dut.imem[1] = 16'h4083;
// addr2: ADD R3, R1, R2
dut.imem[2] = 16'h0298;
// addr3: SUB R4, R1, R2
dut.imem[3] = 16'h12A0;
// addr4: AND R5, R1, R2
dut.imem[4] = 16'h22A8;
// addr5: OR R6, R1, R2
dut.imem[5] = 16'h32B0;
// addr6: STORE [R0+10], R3
dut.imem[6] = 16'h60CA;
// addr7: LOAD R7, R0, 10
dut.imem[7] = 16'h51CA;
// addr8: BEQ R3, R7, 2      (R3==R7 -> PC = PC+1+2 = 11)
dut.imem[8] = 16'h77C2;
// addr9: ADDI R1, R0, 9      (must be skipped by the branch)
dut.imem[9] = 16'h4049;
// addr10: ADDI R1, R0, 10     (must be skipped by the branch)
dut.imem[10] = 16'h404A;
// addr11: JMP 13
dut.imem[11] = 16'h800D;
// addr12: ADDI R1, R0, 11     (dead code, must be skipped by the jump)
dut.imem[12] = 16'h404B;
// addr13: HALT
dut.imem[13] = 16'hF000;

// ---- Reset ----
rst_n = 0;
#12;
rst_n = 1;

// ---- Run until the processor halts (or a safety timeout) ----
for (i = 0; i < 100 && !halted; i = i + 1) begin
    @(posedge clk);
    $display("t=%0t PC=%0d instr=0x%04h halted=%b", $time, pc_out, dut.instr, halted);
end
#10;

// ---- Self-check final architectural state against the expected trace ----
check16(dut.rf.regs[1], 16'd5, "R1 (ADDI result)");
check16(dut.rf.regs[2], 16'd3, "R2 (ADDI result)");
check16(dut.rf.regs[3], 16'd8, "R3 (ADD result)");
check16(dut.rf.regs[4], 16'd2, "R4 (SUB result)");
check16(dut.rf.regs[5], 16'd1, "R5 (AND result)");
check16(dut.rf.regs[6], 16'd7, "R6 (OR result)");
check16(dut.rf.regs[7], 16'd8, "R7 (LOAD result)");
check16(dut.dmem[10], 16'd8, "DMEM[10] (STORE result)");
check16({4'b0,pc_out}, 16'd13, "final PC (post JMP)");
check16({15'b0,halted}, 16'd1, "halted flag");

if (errors == 0)
    $display("\nALL CHECKS PASSED. Simulation complete.");
else
    $display("\n%0d CHECK(S) FAILED. Simulation complete.", errors);

$finish;
end
endmodule

```

3. Screenshots of Output / Waveforms

riscp16_tb.v was run in Vivado for 1000ns. The waveform below shows clk, rst_n, pc_out, halted, and instr stepping through addresses 0-8, then jumping to 11 (BEQ taken) and 13 (post-JMP), matching the expected trace. The console log confirms every self-check passed, including the final register values, the STORE/LOAD round-trip through data memory, the post-branch and post-jump PC, and the halted flag.

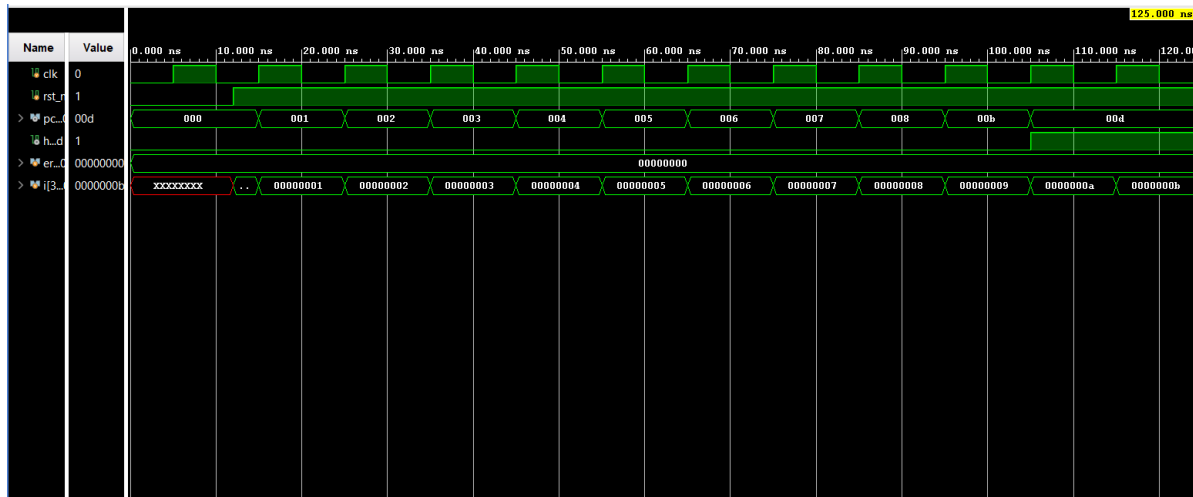


Figure 2: Vivado simulation waveform across the test program

```
# run 1000ns
t=15000 PC=0 instr=0x4045 halted=0
t=25000 PC=1 instr=0x4083 halted=0
t=35000 PC=2 instr=0x0298 halted=0
t=45000 PC=3 instr=0x12a0 halted=0
t=55000 PC=4 instr=0x22a8 halted=0
t=65000 PC=5 instr=0x32b0 halted=0
t=75000 PC=6 instr=0x60ca halted=0
t=85000 PC=7 instr=0x51ca halted=0
t=95000 PC=8 instr=0x77c2 halted=0
t=105000 PC=11 instr=0x800d halted=0
t=115000 PC=13 instr=0xf000 halted=1
CHECK PASSED: R1 (ADDI result) = 5
CHECK PASSED: R2 (ADDI result) = 3
CHECK PASSED: R3 (ADD result) = 8
CHECK PASSED: R4 (SUB result) = 2
CHECK PASSED: R5 (AND result) = 1
CHECK PASSED: R6 (OR result) = 7
CHECK PASSED: R7 (LOAD result) = 8
CHECK PASSED: DMEM[10] (STORE result) = 8
CHECK PASSED: final PC (post JMP) = 13
CHECK PASSED: halted flag = 1

ALL CHECKS PASSED. Simulation complete.
$finish called at time : 125 ns : File "C:/Users/VICTUS/Downloads/files 2/riscp16_tb.v" Line 90
INFO: [USF-XSim-96] XSim completed. Design snapshot 'riscp16_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:10 . Memory (MB): peak = 1719.254 ; gain = 17.785
```

Figure 3: Self-checking testbench console output -- ALL CHECKS PASSED at 125 ns